Trabajo Práctico Grupal AyED

K1051

Apellido Y Nombre	Correo institucional (FRBA)	Usuario de GitHub	Legajo
Fernandez Federico	fedfernandez@frba.utn.edu.ar	fedfernandez	235.036-7
Ocello Santiago	socello@frba.utn.edu.ar	SantiOcello	222.556-6
Cisneros Victoria	vcisneros@frba.utn.edu.ar	Kaitonline	222.224-3

1. Descripción del problema.

Los drones militares siguen una ruta automatizada por GPS, pero el enemigo puede bloquear esa señal. Para ese caso se necesita un sistema de guía de apoyo basado en una tabla de 200 x 200 casilleros. Cada casillero puede contener una orden: despegar, aterrizar, soltar granadas, hacer un ataque kamikaze, esperar y moverse a otro casillero. Nuestro programa en C++ permite cargar, validar, crear, corregir, guardar y visualizar esos ataques para probar la lógica antes de instalarla en los drones. Se maneja con un menú por consola con las siete operaciones que pide la consigna.

2. Descripción de la solución.

2.1 Estructuras de datos

- Orden: una orden dentro de la tabla en memoria (espera, soltarGranada1, soltarGranada2, ataqueKamikaze, aterrizaje, despegue, siguientex, siguientey). Su posición está dada por los índices de la matriz.
- OrdenArchivo: igual a Orden pero con las coordenadas x e y. Es el registro que se guarda en los archivos binarios, donde solo se guardan las celdas que tienen una orden.
- memoria[200][200]: la tabla del dron, de tipo Orden. Se indexa [x][y].
- ocupado[200][200]: matriz paralela de bool que indica qué casilleros tienen una orden cargada. Hace falta porque una Orden vacía (todo en cero) es indistinguible de un casillero sin cargar.
- Ambas matrices se declaran static en main porque pesan cerca de 800 KB y, como variables locales, podrían desbordar la pila.

2.2 Hipótesis tenidas en cuenta

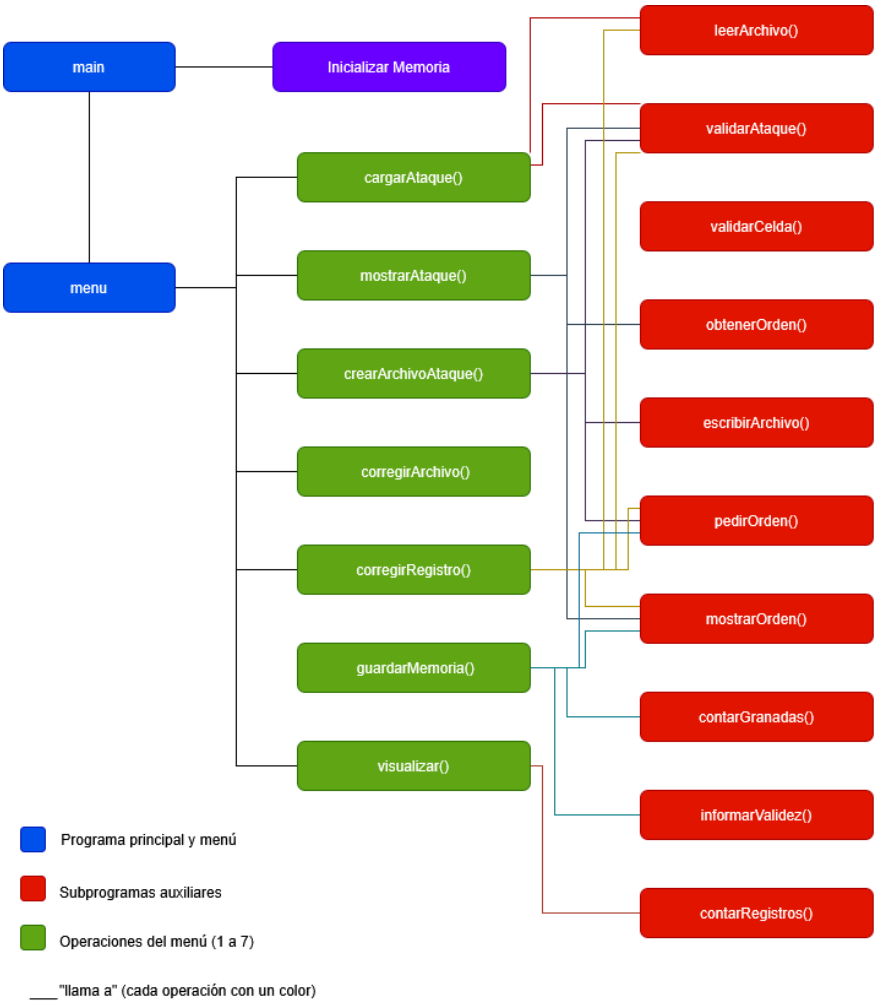
1. Las coordenadas válidas van de 0 a 199 en ambos ejes.
2. Los registros de un archivo pueden estar en cualquier orden. La ruta se reconstruye siguiendo los campos siguientex y siguientey desde el despegue.
3. Celdas aledañas: las 8 vecinas (arriba, abajo, izquierda, derecha y las cuatro diagonales).
4. Una celda no puede apuntar a sí misma. Esperar se modela con el campo espera de la celda y no con un registro adicional.
5. Aterrizar o hacer un ataque kamikaze termina el recorrido: no puede haber ninguna celda después del final. Ambas acciones en la misma celda se consideran un error.
6. Granadas: el dron lleva como máximo dos, la granada 1 y la granada 2, y cada una se suelta una sola vez en todo el recorrido (en la misma celda o en celdas distintas).
7. La espera se expresa en segundos.
8. Si un archivo no cumple alguna regla, se informa el motivo y no se carga. La memoria que ya estaba cargada no se modifica.
9. Los registros que no forman parte de la ruta entre el despegue y el final se consideran un error (la ruta debe estar completa).
10. Los archivos binarios deben tener un tamaño múltiplo del tamaño de OrdenArchivo; si no, se consideran dañados.

11. La celda de despegue no puede soltar granadas ni hacer kamikaze. Sí puede tener espera.
12. En las operaciones 4 y 5 el usuario vuelve a ingresar todos los datos del registro y este se reemplaza. Cada celda se controla al ingresarla (siguiente aledaña, granadas no repetidas). Como corregir una ruta puede requerir cambiar varios registros, el reemplazo se aplica aunque el ataque quede inválido por un momento, y el programa avisa el motivo.
13. Las operaciones 2 y 6 trabajan en orden de recorrido (despegue hasta el final). La operación 6 pide confirmación si el ataque en memoria no es válido o si el archivo ya existe.

2.3 Diagrama de bloques de subprogramas

main() inicializa la memoria y llama al menú, que según la opción elegida deriva a cada una de las siete operaciones. Las operaciones reutilizan subprogramas auxiliares. Además, validarAtaque y pedirOrden llaman a validarCelda; escribirArchivo llama a obtenerOrden; leerArchivo llama a inicializarMemoria; e informarValidez llama a validarAtaque. Toda la entrada por teclado (menú y operaciones) pasa por leerLinea, leerEntero y leerSiNo, que no se dibujan para no cargar el diagrama.

Diagrama de bloques de subprogramas



2.4 División de tareas del equipo

Integrante	Tareas a Cargo
Victoria Cisneros	Estructuras de datos y Operación 1 (carga y validaciones)
Victoria Cisneros	Operación 2 (mostrar con paginación)
Federico Fernandez	Operación 3 (crear archivo)
Santiago Ocello	Operaciones 4 y 5 (corregir en archivo y en memoria)
Federico Fernandez	Operaciones 6 y 7 (guardar y visualizar en HTML)
Victoria Cisneros, Federico Fernandez y Santiago Ocello	Pruebas, informe y repositorio

3. Descripción de las operaciones.

Operación 1 - Cargar archivo de ataque en memoria. Se lee el archivo a una tabla temporal. Si todo es válido, recién ahí se copia a la memoria del dron. Las reglas se controlan en validarCelda y validarAtaque:

- Coordenadas dentro de la tabla (0 a 199) y sin celdas repetidas en el archivo.
- Una única instrucción de despegue, sin granadas ni kamikaze en esa celda.
- Una única instrucción de aterrizaje o kamikaze, que termina el recorrido.
- Cada granada (1 y 2) se suelta como máximo una vez.
- Ruta completa y sin huecos: cada siguiente está dentro de la tabla, tiene registro y es una de las 8 celdas aledañas.
- No se pasa dos veces por la misma coordenada.
- No quedan registros sueltos fuera de la ruta.

Operación 2 - Mostrar ataque. Muestra el ataque en el orden en que lo recorre el dron, del despegue al final. Cada línea indica la celda, sus acciones en el orden de precedencia (despegue, aterrizaje, granada 1, granada 2, kamikaze, espera) y la celda siguiente. Cada 20 líneas se detiene hasta que el usuario presiona una tecla. Si no hay nada cargado, lo informa.

Operación 3 - Crear un archivo de ataque nuevo. Pide la ruta del archivo (y confirma si ya existe) y la celda de despegue. Después pide, celda por celda, todas las acciones en el orden de precedencia y la celda siguiente. Cada celda se controla al ingresarla: despegue sin ataques, siguiente aledaña, granadas no repetidas y sin pasar dos veces por la misma celda. Si algo no se cumple, vuelve a pedir los datos. Aterrizar o hacer kamikaze termina el recorrido. El archivo se escribe recién al final, ya validado.

Operación 4 - Corregir un registro del archivo. Pide la ruta del archivo y las coordenadas x e y. Busca el registro, lo muestra y, si el usuario acepta, pide todos los datos de nuevo y lo reemplaza en el mismo lugar del archivo. Si la celda no tiene registro, no modifica nada. Después vuelve a validar el archivo y avisa si todavía tiene errores.

Operación 5 - Corregir un registro en memoria. Igual a la Operación 4, pero sobre la tabla en memoria. Después de reemplazar el registro informa si el ataque quedó válido o cuál es el error que falta corregir.

Operación 6 - Guardar memoria en un archivo nuevo. Guarda en un archivo nuevo los registros de la memoria, en orden de recorrido. Si la memoria está vacía no crea nada. Si el ataque en memoria no es válido, informa el motivo y pide confirmación. Si el archivo ya existe, pregunta si se sobrescribe.

Operación 7 - Visualizar en HTML. Genera una tabla HTML de 200 x 200 con el ataque en memoria (columnas = x, filas = y). Cada celda con una orden se pinta según su acción (despegue, aterrizaje, kamikaze, granada, espera o movimiento) y muestra sus coordenadas al pasar el mouse. Si el nombre no termina en .html, se lo agrega.

Menú. Toda la entrada se lee por líneas y se valida: si el usuario escribe una letra o un número fuera de rango, el programa vuelve a pedir el dato.

4. Casos de prueba.

4.1 Tabla de casos

Nro.	Caso	Archivo	Resultado esperado
1	Ruta válida con diagonal y una espera	ok_diag.dat	Cargado correctamente (3 registros)
2	Granada 1, granada 2, espera y aterrizaje	ok_granadas.dat	Cargado correctamente (4 registros)
3	Registros desordenados dentro del archivo	desordenado.dat	Cargado; la Op. 2 los muestra en orden de recorrido
4	Dos registros para la misma celda	prueba2_repetida.dat	Rechaza: celda repetida
5	Coordenada fuera de la tabla (300, 300)	fuera_coord.dat	Rechaza: fuera de la tabla (0 a 199)
6	Siguiente fuera de la tabla (-1, 0)	fuera_siguiente.dat	Rechaza: apunta fuera de la tabla
7	Registro suelto fuera de la ruta	huerfano.dat	Rechaza: registros que no forman parte de la ruta
8	Archivo truncado	truncado.dat	Rechaza: no corresponde a registros completos
9	Archivo vacío	vacio.dat	Rechaza: no tiene registros
10	Sin despegue o dos despegues	sin_despegue.dat, dos_despegues.dat	Rechaza: debe haber exactamente 1 despegue
11	Despegue con granada	despegue_ataca.dat	Rechaza: el despegue no puede tener ataques
12	Sin final o dos finales	sin_final.dat, dos_finales.dat	Rechaza: exactamente 1 aterrizaje o kamikaze
13	Aterrizaje y kamikaze en la misma celda	aterr_y_kami.dat	Rechaza: aterrizaje y kamikaze a la vez
14	Celda que se apunta a sí misma	se_apunta.dat	Rechaza: apunta a sí misma
15	Salto a una celda no aledaña	salto.dat	Rechaza: no son celdas aledañas
16	Hueco en la ruta	hueco.dat	Rechaza: el siguiente no tiene registro
17	Ruta que vuelve a pasar por una celda	ciclo.dat	Rechaza: pasa más de una vez por la celda
18	La misma granada en dos celdas	dos_granada1.dat, dos_granada2.dat	Rechaza: máximo 1
19	Carga fallida después de una carga correcta	ok_diag.dat y luego hueco.dat	La memoria anterior se conserva (opción 2)
20	Op. 2 con la memoria vacía	-	Informa que no hay ataque cargado

21	Op. 2 con una ruta de 45 celdas	larga45.dat	Se detiene cada 20 líneas hasta mostrar los 45 pasos
22	Op. 3 con datos mal escritos (letras, fuera de rango, distinto de s o n)	-	Los vuelve a pedir
23	Op. 3 con celda ya usada o siguiente no aledaña	-	Rechaza y vuelve a pedir el registro
24	Op. 3 con una granada ya soltada	-	No la vuelve a ofrecer
25	Op. 3 completa (despegue, granada 1, granada 2, kamikaze)	-	Archivo creado; se carga sin errores con la Op. 1
26	Op. 4 con una celda que no existe en el archivo	ok_diag.dat	Informa y no modifica el archivo
27	Op. 4 cambiando la espera de un registro	copia de ok_diag.dat	Registro reemplazado y archivo válido
28	Op. 5 cambiando la espera de una celda	-	El ataque queda válido
29	Op. 5 apuntando a una celda aledaña sin registro	-	Avisa el hueco
30	Op. 6: memoria vacía / archivo existente / guardado	-	No guarda / pregunta si sobrescribe / guarda en orden de recorrido
31	Op. 7 (HTML)	-	Archivo HTML que se abre en el navegador
32	Menú con opción inválida o letra	-	Vuelve a pedirla; el programa no se cuelga

Disclaimer: Los Nros pintados en amarillo representan los casos de ejemplos del apartado 4.2 con captura.

4.2 Ejemplos de salida de algunos casos (con captura)

- **Ejemplo 1.** Carga de una ruta válida y visualización con la Operación 2 (casos 1 y 3).

Caso 1:

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\ok_diag.dat
Archivo cargado y validado correctamente (3 registros).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 2

ATAQUE CARGADO EN MEMORIA (3 registros, en orden de recorrido)

Paso 1: (x=1, y=1) Despegue + Mover a (2, 1)
Paso 2: (x=2, y=1) Espera 5 s + Mover a (3, 2)
Paso 3: (x=3, y=2) Kamikaze

Fin del ataque.
```

Caso 3:

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\desordenado.dat
Archivo cargado y validado correctamente (3 registros).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 2

ATAQUE CARGADO EN MEMORIA (3 registros, en orden de recorrido)

Paso 1: (x=1, y=1) Despegue + Mover a (2, 1)
Paso 2: (x=2, y=1) Espera 5 s + Mover a (3, 2)
Paso 3: (x=3, y=2) Kamikaze

Fin del ataque.
```

- **Ejemplo 2.** Tres archivos inválidos son rechazados con el motivo (casos 4, 16 y 18).

Caso 4:

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\prueba2_repetida.dat
Archivo NO cargado. Registro 3: la celda [2][2] esta repetida en el archivo.
```

Caso 16:

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\hueco.dat
Archivo NO cargado. Hueco en la ruta: [1][1] apunta a [2][1], que no tiene registro.
```

Caso 18:

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\dos_granada2.dat
Archivo NO cargado. La granada 2 se suelta en 2 celdas (maximo 1).
```

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\dos_granada1.dat
Archivo NO cargado. La granada 1 se suelta en 2 celdas (maximo 1).
```

- **Ejemplo 3.** Creación de un archivo con la Operación 3, con un dato mal escrito, una celda repetida rechazada, y luego carga y visualización del archivo creado (casos 22, 23, 24 y 25).

```
Ingrese una opcion: 3

Ingrese ruta y nombre del archivo a crear: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\nuevoej.dat
El archivo ya existe. Desea sobrescribirlo (s/n): s
Celda de despegue del dron
Coordenada x: abc
Valor invalido. Ingrese un numero entre 0 y 199.
Coordenada x: 10
Coordenada y: 10

Registro 1 - celda [10][10]
Celda de despegue: no puede soltar granadas ni hacer kamikaze.
Aterriza en esta celda (s/n): n
Segundos de espera (0 si no espera): 0
Siguiente x: 11
Siguiente y: 10

Registro 2 - celda [11][10]
Aterriza en esta celda (s/n): n
Suelta la granada 1 (s/n): s
Suelta la granada 2 (s/n): n
Hace ataque kamikaze (s/n): n
Segundos de espera (0 si no espera): 3
Siguiente x: 12
Siguiente y: 11

Registro 3 - celda [12][11]
Aterriza en esta celda (s/n): n
(La granada 1 ya se suelta en otra celda.)
Suelta la granada 2 (s/n): s
Hace ataque kamikaze (s/n): n
Segundos de espera (0 si no espera): 0
```

```
Siguiente x: 11
Siguiente y: 10
Error: la celda [11][10] ya forma parte de la ruta (no se puede pasar dos veces). Vuelva a ingresar los datos.
Aterriza en esta celda (s/n): n
(La granada 1 ya se suelta en otra celda.)
Suelta la granada 2 (s/n): s
Hace ataque kamikaze (s/n): n
Segundos de espera (0 si no espera): 0
Siguiente x: 13
Siguiente y: 12

Registro 4 - celda [13][12]
Aterriza en esta celda (s/n): n
(La granada 1 ya se suelta en otra celda.)
(La granada 2 ya se suelta en otra celda.)
Hace ataque kamikaze (s/n): s
Segundos de espera (0 si no espera): 0

Archivo creado con exito. Total registros: 4

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\nuevoej.dat
Archivo cargado y validado correctamente (4 registros).
```

```
1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 2

ATAQUE CARGADO EN MEMORIA (4 registros, en orden de recorrido)

Paso 1: (x=10, y=10) Despegue + Mover a (11, 10)
Paso 2: (x=11, y=10) Granada 1 + Espera 3 s + Mover a (12, 11)
Paso 3: (x=12, y=11) Granada 2 + Mover a (13, 12)
Paso 4: (x=13, y=12) Kamikaze

Fin del ataque.

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 8

Saliendo...
```

- **Ejemplo 4.** Corrección de un registro en memoria (caso 28).

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\ok_diag.dat
Archivo cargado y validado correctamente (3 registros).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 5

Coordenada x del registro a corregir: 2
Coordenada y del registro a corregir: 1
Registro actual: (x=2, y=1) Espera 5 s + Mover a (3, 2)
Desea ingresar nuevamente los datos (s/n): s
Es la celda de despegue (s/n): n
Aterriza en esta celda (s/n): n
Suelta la granada 1 (s/n): n
Suelta la granada 2 (s/n): n
Hace ataque kamikaze (s/n): n
Segundos de espera (0 si no espera): 9
Siguiente x: 3
Siguiente y: 2
Registro reemplazado.
El ataque quedo valido.

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 2

ATAQUE CARGADO EN MEMORIA (3 registros, en orden de recorrido)

Paso 1: (x=1, y=1) Despegue + Mover a (2, 1)
Paso 2: (x=2, y=1) Espera 9 s + Mover a (3, 2)
Paso 3: (x=3, y=2) Kamikaze

Fin del ataque.
```

- **Ejemplo 5.** Corrección en archivo de una celda que no existe: no se modifica nada (caso 26).

```
Ingrese una opcion: 4

Ingrese la ruta y nombre del archivo a corregir: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\ok_diag.dat
Coordenada x del registro a corregir: 99
Coordenada y del registro a corregir: 99
El archivo no tiene ningun registro en la celda [99][99].
```

- **Ejemplo 6:** Sin despegue o despegue con ataque (caso 10, caso 11).

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\sin_despegue.dat
Archivo NO cargado. Debe haber exactamente 1 despegue (hay 0).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\despegue_ataca.dat
Archivo NO cargado. El despegue en [1][1] no puede estar acompanado de ataques.
```

- **Ejemplo 7:** Sin final o con dos finales (caso 12).

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\dos_finales.dat
Archivo NO cargado. Debe haber exactamente 1 aterrizaje o kamikaze (hay 2).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\sin_final.dat
Archivo NO cargado. Debe haber exactamente 1 aterrizaje o kamikaze (hay 0).
```

- **Ejemplo 8:** salto a una celda lejana (caso 15).

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salto.dat
Archivo NO cargado. Salto invalido de [1][1] a [5][1]: no son celdas aledanas.
```

- **Ejemplo 9:** ruta que repite una celda (caso 17).

```
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\ciclo.dat
Archivo NO cargado. La ruta pasa mas de una vez por [2][1].
```

- **Ejemplo 10:** Se detiene cada 20 líneas hasta mostrar los 45 pasos (caso 21):

```
Ingrese una opcion: 2

No hay ningun ataque cargado en memoria.

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\larga45.dat
Archivo cargado y validado correctamente (45 registros).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 2

ATAQUE CARGADO EN MEMORIA (45 registros, en orden de recorrido)

Paso 1: (x=1, y=1) Despegue + Mover a (2, 1)
Paso 2: (x=2, y=1) Mover a (3, 1)
Paso 3: (x=3, y=1) Mover a (4, 1)
Paso 4: (x=4, y=1) Mover a (5, 1)
Paso 5: (x=5, y=1) Mover a (6, 1)
Paso 6: (x=6, y=1) Mover a (7, 1)
Paso 7: (x=7, y=1) Mover a (8, 1)
Paso 8: (x=8, y=1) Mover a (9, 1)
Paso 9: (x=9, y=1) Mover a (10, 1)
Paso 10: (x=10, y=1) Mover a (11, 1)
Paso 11: (x=11, y=1) Mover a (12, 1)
Paso 12: (x=12, y=1) Mover a (13, 1)
Paso 13: (x=13, y=1) Mover a (14, 1)
Paso 14: (x=14, y=1) Mover a (15, 1)
Paso 15: (x=15, y=1) Mover a (16, 1)
Paso 16: (x=16, y=1) Mover a (17, 1)
Paso 17: (x=17, y=1) Mover a (18, 1)
Paso 18: (x=18, y=1) Mover a (19, 1)
Paso 19: (x=19, y=1) Mover a (20, 1)
Paso 20: (x=20, y=1) Mover a (21, 1)

-- Presione una tecla para ver la siguiente pagina --
```

- **Ejemplo 11:** operación 6 con memoria vacía, guardar una copia de un archivo con op6, luego de realizar una copia del archivo querer hacer uno nuevo con el mismo nombre y rechazarlo (caso 30):

```
Ingrese una opcion: 6

La memoria esta vacia, no hay nada para guardar.

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 1

Ingrese la ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\ok_diag.dat
Archivo cargado y validado correctamente (3 registros).

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 6

Ingrese ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salida_copia.dat
Memoria guardada con exito. Total registros: 3

1- Cargar archivo de ataque en memoria.
2- Mostrar un ataque cargado.
3- Crear un archivo de ataque nuevo.
4- Corregir un registro del archivo.
5- Corregir un registro en memoria.
6- Guardar memoria en un archivo nuevo.
7- Visualizar un archivo de ataque en html.
8- Salir.
Ingrese una opcion: 6

Ingrese ruta y nombre del archivo: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salida_copia.dat
El archivo ya existe. Desea sobrescribirlo (s/n): n
No se guardo el archivo.
```

Luego, cambia la espera de un registro (caso 27) en el archivo copia.

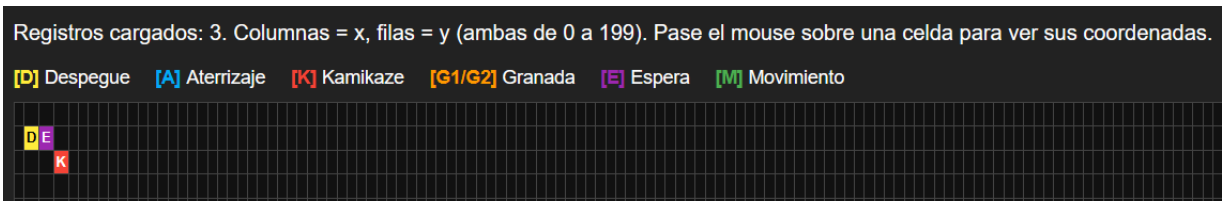
```
Ingrese una opcion: 4

Ingrese la ruta y nombre del archivo a corregir: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salida_copia.dat
Coordenada x del registro a corregir: 2
Coordenada y del registro a corregir: 1
Registro actual: (x=2, y=1) Espera 5 s + Mover a (3, 2)
Desea ingresar nuevamente los datos (s/n): s
Es la celda de despegue (s/n): n
Aterriza en esta celda (s/n): n
Suelta la granada 1 (s/n): n
Suelta la granada 2 (s/n): n
Hace ataque kamikaze (s/n): n
Segundos de espera (0 si no espera): 9
Siguiente x: 3
Siguiente y: 2
Registro reemplazado.
El archivo quedo valido.
```

Genero el html con la operación 7 (caso 31):

```
Ingrese una opcion: 7

Ingrese ruta y nombre del archivo HTML: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salida_mapa.html
Archivo HTML generado con exito: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\salida_mapa.html
```



- **Ejemplo 12:** Ingresar un valor distinto de 1 a 8 (caso 32).

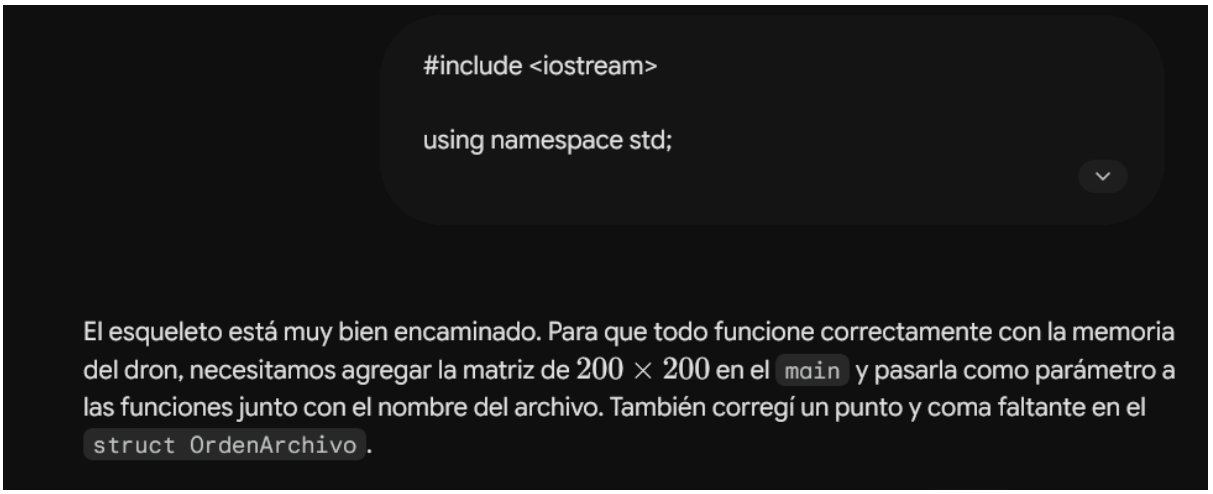
```
Ingrese una opcion: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\dos_granada2.dat
Valor invalido. Ingrese un numero entre 1 y 8.
Ingrese una opcion: C:\Users\Victoria\Desktop\AyED\tp-ayed-drones\pruebas\dos_granada1.dat
Valor invalido. Ingrese un numero entre 1 y 8.
```

5. Herramientas utilizadas.

5.1 Gemini:

URL: https://gemini.google.com

Le envié el esqueleto de la implementación del menú y la operación 1 y operación 2, y me tiró una posibilidad de menú y de implementaciones de las respectivas operaciones.



Pedí ayuda para inicializar la matriz de forma limpia, no lo había tenido en cuenta y fue una corrección de la misma.

```
// Limpia la matriz antes de cargar un archivo nuevo
void inicializarMemoria(Orden memoria[200][200]) {
    for (int i = 0; i < 200; i++) {
        for (int j = 0; j < 200; j++) {
            memoria[i][j] = Orden();
        }
    }
}
```

Mande esto como el código del menú, y corrigió la parte del switch junto con los prototipos de la función.

```
void menu(char nombreArchivo[]) {
    int opcion;
    do {
        cout << "1- Cargar archivo de ataque en memoria." << endl;
        cout << "2- Mostrar un ataque cargado." << endl;
        cout << "3- Crear un archivo de ataque nuevo." << endl;
        cout << "4- Corregir un registro del archivo." << endl;
        cout << "5- Corregir un registro en memoria." << endl;
        cout << "6- Guardar memoria en un archivo nuevo." << endl;
        cout << "7- Visualizar un archivo de ataque en html." << endl;
        cout << "8- Salir." << endl;
        cout << "Ingrese una opcion: ";
        cin >> opcion;

        switch (opcion) {
            case 1: cargarAtaque(nombreArchivo); break;
            case 2: mostrarAtaque(nombreArchivo); break;
            case 3: crearArchivoAtaque(nombreArchivo); break;
            case 4: corregirArchivo(nombreArchivo); break;
            case 5: corregirRegistro(nombreArchivo); break;
            case 6: guardarMemoria(nombreArchivo); break;
            case 7: visualizar(nombreArchivo); break;
            case 8: cout << "Saliendo..." << endl; break;
            default: cout << "Ingresaste una opcion invalida." << endl;
        }
    } while (opcion != 8);
}
```

Mejora de los prototipos y la función menú

```
// ---------- PROTOTIPOS ----------
void inicializarMemoria(Orden memoria[][COLUMNAS], bool hay[][COLUMNAS]);
void menu(Orden memoria[][COLUMNAS], bool hay[][COLUMNAS]);
void cargarAtaque(Orden memoria[][COLUMNAS], bool hay[][COLUMNAS]);
void mostrarAtaque(const Orden memoria[][COLUMNAS], const bool hay[][COLUMNAS]);
void crearArchivoAtaque();
void corregirArchivo();
void corregirRegistro(Orden memoria[][COLUMNAS], bool hay[][COLUMNAS]);
void guardarMemoria(const Orden memoria[][COLUMNAS], const bool hay[][COLUMNAS]);
void visualizar(const Orden memoria[][COLUMNAS], const bool hay[][COLUMNAS]);
```

Tome este consejo para el main para evitar errores

```
  // static: la tabla pesa ~800 KB y como variable local normal puede
  // desbordar la pila y cerrar el programa.
  static Orden memoria[FILAS][COLUMNAS];
  static bool hay[FILAS][COLUMNAS];
```

En la operación 2 utilizó esta implementación del cin.ignore y cin.get recomendado por la IA.

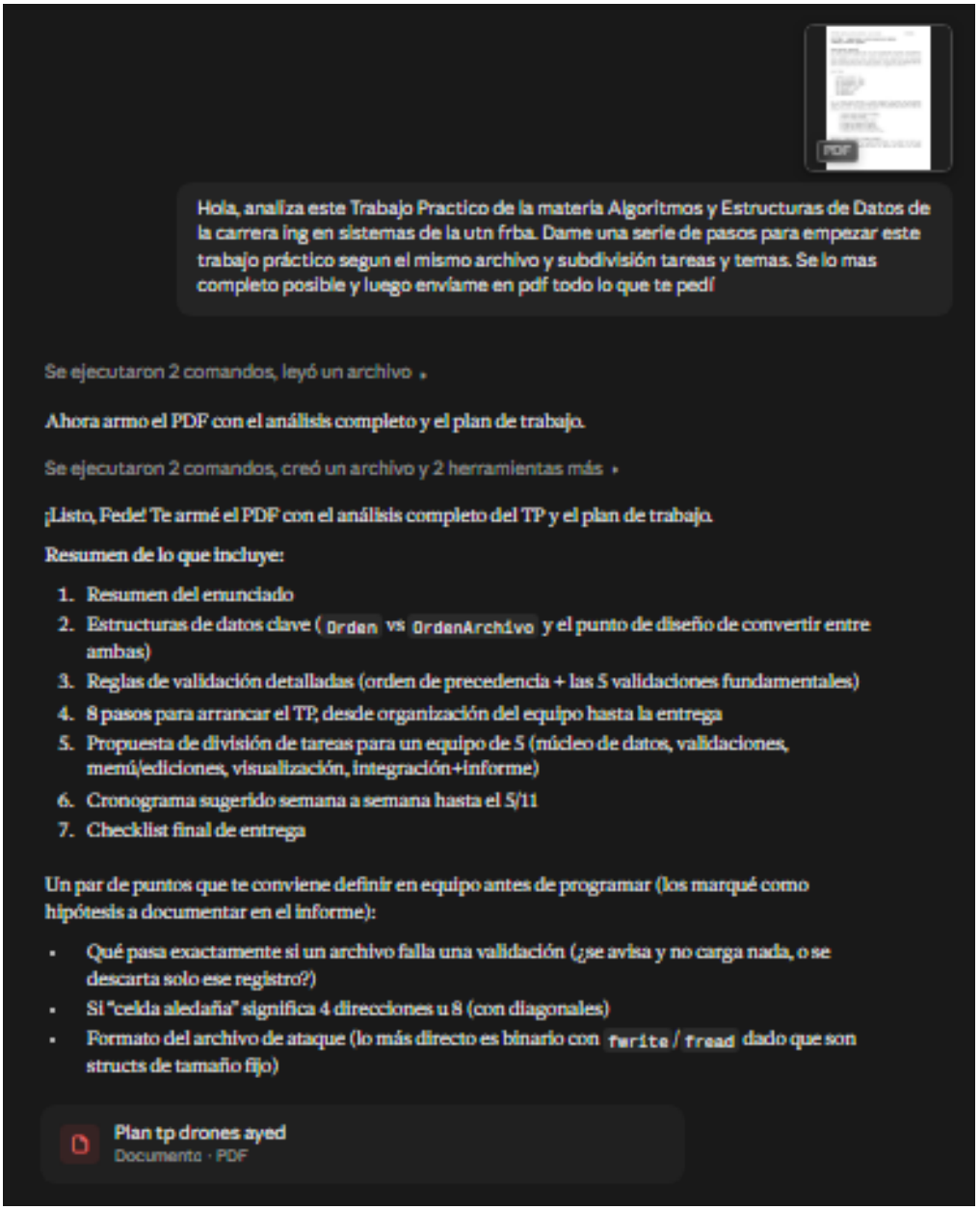
```
    if (lineasImpresas >= 20) {
        cout << "\nPresione ENTER para mostrar la siguiente pagina..."; //[cite: 4,

        cin.ignore(); // Limpia cualquier salto de linea residual en el buffer
        cin.get();    // Frena la ejecucion hasta que el usuario presione ENTER

        lineasImpresas = 0;
    }
}
```

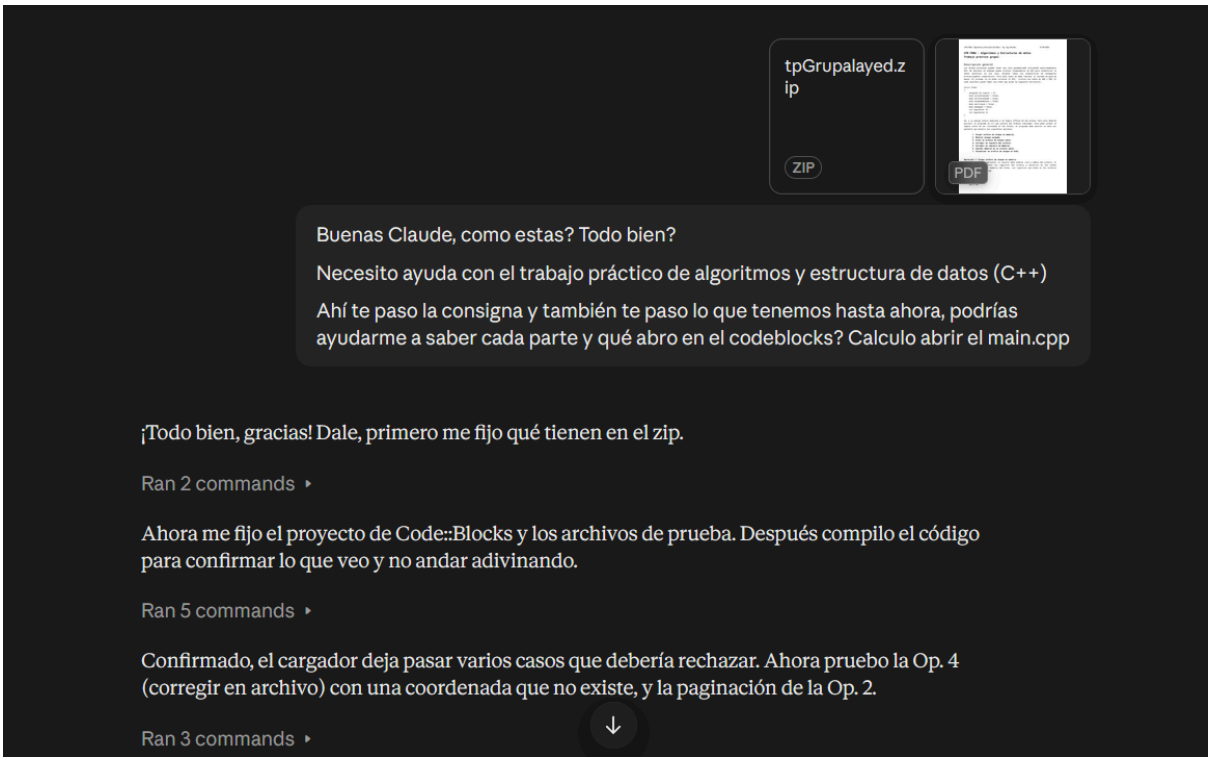
5.2 Claude:

URL: https://claude.ai



Le envié el TP a claude para organizar el trabajo por pasos y simplificar algunas cuestiones para empezar.

Inicio de la conversación. Le entregamos la consigna (PDF) y el zip con el código que teníamos. Claude abrió el proyecto, lo compiló y ejecutó pruebas antes de responder; esos pasos aparecen como Ran 2 commands, Ran 5 commands y Ran 3 commands.



Primer diagnóstico. Explica qué abrir en Code::Blocks, avisa que conio.h solo existe en Windows y arma un mapa de cada operación de la consigna contra nuestras funciones, con su estado.

Todo bien, gracias. Revisé el zip, compilé el código y probé cada operación con archivos de prueba.

Qué abrir en Code::Blocks

Abrí `Trabajo practico AyED.cbp` (doble clic) y no el `main.cpp` suelto. Todo el código está en `main.cpp` (858 líneas), pero el `.cbp` trae la configuración de compilación. Un detalle: el proyecto tiene un tercer target, `prueba.parcial`, que está configurado como DLL. Dejá seleccionado Debug, porque con ese target no se genera un `.exe`.

Además, `conio.h` (el `getch()`) solo existe en Windows con MinGW. Si alguien del grupo usa Mac o Linux, no le va a compilar.

Mapa de la consigna contra el código

Op.	Función	Estado
1. Cargar	`cargarAtaque`	Anda, pero valida de menos (ver abajo)
2. Mostrar	`mostrarAtaque`	Anda, con un detalle en la paginación
3. Crear archivo	`crearArchivoAtaque`	Tiene un bug de diseño
4. Corregir en archivo	`corregirArchivo`	Se cae (desborda un arreglo)
5. Corregir en memoria	`corregirRegistro`	Se aparta de lo que pide la consigna
6. Guardar	`guardarMemoria`	Bien
7. HTML	`visualizar`	Bien, incluso más vistoso de lo pedido

Errores encontrados al ejecutar el código. La Operación 4 se cae por un desborde de arreglo (el bucle llega hasta FILAS inclusive y el arreglo local pesa 1,1 MB en la pila). Las Operaciones 4 y 5 no reemplazan el registro completo como pide la consigna. En la Operación 3, la opción Esperar crea dos registros en la misma celda. La Operación 1 aceptaba celdas repetidas, coordenadas fuera de rango, registros sueltos fuera de la ruta y un siguientex negativo que leía fuera del arreglo.

Lo que encontré (lo probé corriendo el programa)

Op. 4 (urgente). El bucle `for (x=1; x<=FILAS; ...)` escribe en `registro[200]`, que está fuera del arreglo. Me dio *stack smashing* y *segfault*. Además, `registro[FILAS][COLUMNAS]` pesa 1,1 MB en la pila, igual que en `prueba()`. Y si la coordenada no existe, `nuevo_reg` queda vacío y por lectura del código se escribe en la posición 0 del archivo. Eso último no pude confirmarlo corriendo, porque se cae antes.

Ops. 4 y 5 contra la consigna. La consigna dice que se muestra el registro y el usuario lo **reingresa completo** para reemplazarlo. Ustedes hicieron un mini-menú (espera, granada, aterrizaje) que no permite cambiar `siguientex/y`. Es mucho código y es donde están los bugs. Lo más simple es una función `pedirOrden()` que usen las ops. 3, 4 y 5. Además, la Op. 4 trabaja sobre archivos que pueden estar inválidos justamente para arreglarlos, así que no debería bloquear: que avise y listo.

Op. 3. La opción "Esperar" crea un registro que apunta a sí mismo y después crea **otro registro en la misma celda.** Eso es lo que generó `prueba2`, que tiene dos registros en (2,2). La consigna dice que una celda puede tener varias acciones a la vez (por eso la lista de precedencia), así que por cada celda hay que preguntar todas las acciones y después el siguiente.

Op. 1. Estos casos se aceptan como válidos y no deberían:

- Dos registros en la misma celda: el segundo pisa al primero en silencio (`prueba2` carga "correctamente").
- Coordenadas fuera de 0–199: el registro se ignora sin avisar.
- `siguientex = -1`: lee fuera del arreglo (comportamiento indefinido).
- Un registro suelto fuera de la ruta (huérfano): pasa, y la consigna pide una ruta sin huecos.
- 3 granadas en total: pasa (ver hipótesis).

Paginación y decisiones pendientes. Detecta que la paginación de la Operación 2 pide ENTER dos veces desde la segunda página, y detalles menores como prueba() ejecutándose en cada arranque y la espera en minutos en la Operación 4 y en segundos en la 5. Termina con cuatro hipótesis que el grupo tenía que decidir: celdas aledañas (4 u 8), cantidad de granadas, aterrizaje junto con kamikaze y unidad de la espera.

Op. 2. Después de `cin >> opcion` queda un `\n` en el buffer. Por cómo están armados `cin.ignore(); cin.get();`, de la segunda página en adelante probablemente haya que apretar ENTER dos veces. Con `getch()` queda resuelto.

Detalles menores:

- `prueba()` se ejecuta en cada arranque y pisa el archivo `prueba`. Sacala de `main` para la entrega.
- La Op. 4 dice "minutos" y la Op. 5 dice "segundos".
- Si hay granada y aterrizaje en la misma celda, `menu_archivo` imprime dos opciones "3-".

Hipótesis que tienen que decidir y escribir en el informe

1. ¿"Celdas aledañas" son 4 o 8 vecinas? Su `prueba` usa diagonales.
2. ¿Hay máximo 2 granadas en total, o una de cada tipo? Hoy las ops. 4 y 5 asumen 2 y el cargador no controla nada.
3. ¿Aterrizaje y kamikaze en la misma celda son un error?
4. Unidad de la espera.

Mejoras aplicadas al código

La mejora 1 y la detección de los errores de la Operación 4 se probaron ejecutando el código durante la conversación con Claude. Las mejoras 2 a 8 corresponden a la versión final del programa, tal como la describen el informe del grupo y el resumen del proyecto.

1. Operación 1: validación completa y carga segura

- **Problema.** Aceptaba celdas repetidas (la segunda pisaba a la primera), ignoraba en silencio las coordenadas fuera de 0 a 199, leía fuera del arreglo con un siguiente negativo y no detectaba registros sueltos fuera de la ruta. Además, un archivo inválido dejaba vacía la memoria que ya estaba cargada.
- **Mejora.** El archivo se lee a una tabla temporal y recién después de validarlo se copia a la memoria del dron. La función validarAtaque trabaja en dos pasos. Primero revisa un único despegue sin ataques, un único aterrizaje o kamikaze, y que aterrizaje y kamikaze no coincidan en una celda. Después recorre la ruta desde el despegue y controla que cada siguiente esté dentro de la tabla, no apunte a sí misma, sea aledaña, tenga registro y no se repita. Al final compara las celdas recorridas con los registros totales. También rechaza archivos cuyo tamaño no es múltiplo de un registro, y cada rechazo informa el motivo.
- **Verificación (hecha en la conversación).** 3 archivos válidos se cargaron y 15 inválidos se rechazaron con su mensaje: celda repetida, coordenada fuera de la tabla, siguiente fuera de la tabla, registro suelto, archivo truncado, sin despegue, dos despegues, despegue con ataque, sin final, dos finales, aterrizaje con kamikaze, celda que apunta a sí misma, salto no aledaño, hueco y ciclo. Una carga fallida conservó la memoria anterior. AddressSanitizer y UBSan no reportaron errores.

2. Operación 2: paginación y orden de recorrido

- **Problema.** cin.ignore() y cin.get() pedían ENTER dos veces desde la segunda página, y los registros salían ordenados por coordenada y no por ruta.
- **Mejora.** Pausa cada 20 líneas con getch(), muestra el ataque en orden de recorrido (del despegue al final) con las acciones en el orden de precedencia, y avisa si no hay nada cargado.
- **Verificación.** Archivo de 45 registros que pausa cada 20 líneas. **Mejora 2 (Op. 2):** caso 21.

3. Operación 3: crear un archivo de ataque

- **Problema.** La opción Esperar creaba un registro que apuntaba a sí mismo y después otro en la misma celda; así se generó prueba2, con dos registros en (2,2). Tampoco validaba nada.
- **Mejora.** Con pedirOrden pide, por cada celda, todas las acciones en orden de precedencia (la espera es un campo, no un registro extra) y la celda siguiente. Controla cada celda al ingresarla: despegue sin ataques, siguiente aledaña, granadas no repetidas y ninguna celda usada dos veces. El archivo se escribe recién al final, ya validado.
- **Verificación.** Casos 22 a 25 del informe, con capturas en Windows.

4. Operación 4: corregir un registro del archivo

- **Problema.** El bucle for (x = 1; x <= FILAS; ...) escribía fuera del arreglo (confirmado con AddressSanitizer y con un segmentation fault). El arreglo local pesaba 1,1 MB de pila. Por lectura del código, si la coordenada no existía se escribía un registro vacío en la posición 0. Además usaba un mini-menú en lugar de reingresar el registro completo, que es lo que pide la consigna.
- **Mejora.** Abre el archivo con rb+, busca el registro por x e y, lo muestra, pide todos los datos de nuevo con pedirOrden y lo reemplaza en el mismo lugar con fseek y fwrite, sin cargar el archivo entero. Si la celda no existe, no modifica nada. Después valida el archivo y avisa si todavía tiene errores, sin bloquear el reemplazo, porque corregir una ruta puede exigir cambiar varios registros.
- **Verificación. Mejora 4 (Op. 4):** casos 26 y 27.

5. Operación 5: corregir un registro en memoria

- **Problema.** Tenía el mismo mini-menú, no permitía cambiar siguientex ni siguientey, y con granada y aterrizaje en la misma celda mostraba dos opciones 3.
- **Mejora.** Usa el mismo mecanismo que la Operación 4, pero sobre la tabla. Después del reemplazo informa si el ataque quedó válido o cuál es el error que falta corregir.
- **Verificación. Mejora 5 (Op. 5):** caso 28.

6. Lectura de datos por teclado

- **Problema.** Una letra en el menú dejaba el programa en un bucle infinito.
- **Mejora.** Toda la entrada se lee por líneas con leerLinea, leerEntero y leerSiNo, que vuelven a pedir el dato si no es válido. Si se termina la entrada, el programa sale en lugar de repetir.
- **Verificación. Mejora 6 (letra en el menú):** caso 32.

7. Limpieza general

- prueba() se ejecutaba en cada arranque y sobrescribía el archivo prueba; se sacó de main.
- La espera quedó unificada en segundos (antes era minutos en las Operaciones 3 y 4 y segundos en la 5).
- Las dos matrices de unos 800 KB son static en main y ningún arreglo grande queda en la pila de una función. Antes había 1,1 MB en corregirArchivo y en prueba.
- Dejaron de existir ContarGranadas y menu_archivo: su lógica pasó a validarAtaque y pedirOrden.
- El programa compila sin warnings con -Wall -Wextra -pedantic.

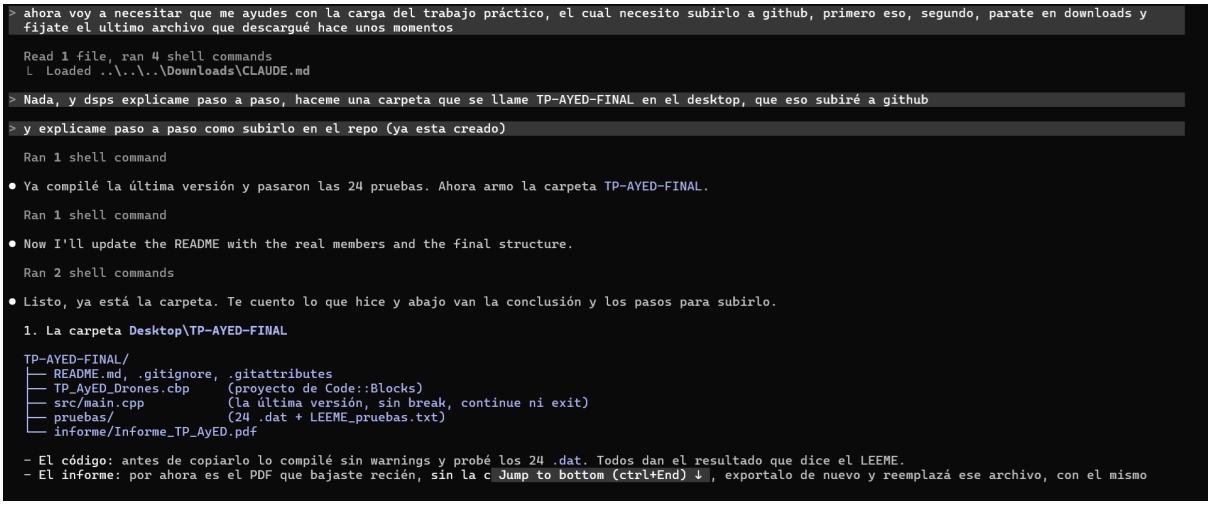
8. Archivos de prueba

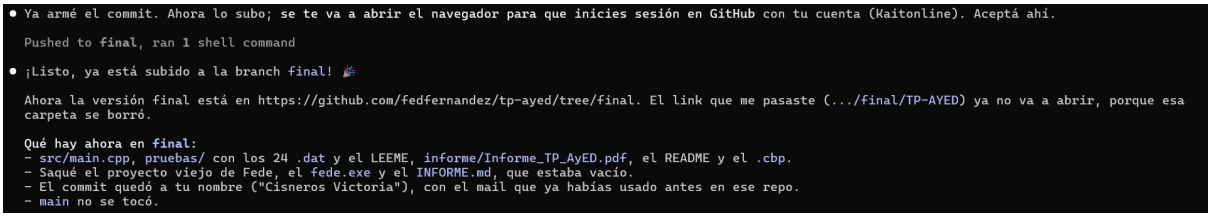
La carpeta pruebas/ tiene 24 archivos .dat y un LEEME_pruebas.txt con el resultado esperado de cada uno en la opción 1. Según el resumen del proyecto, la batería completa de la Operación 1 pasa con AddressSanitizer, UBSan y con la pila limitada a 1 MB.

5.3 Claude Code:

URL: https://claude.com/es/product/claude-code

Le pedimos a Claude Code que nos ayude a subir la versión final del trabajo práctico a GitHub.





6. Conclusión.

Con este trabajo práctico logramos desarrollar un sistema de guía de apoyo para drones, que permite cargar, validar, crear, corregir, guardar y visualizar ataques sobre una tabla de 200 x 200 casilleros, desde un menú por consola con las siete operaciones pedidas. Pudimos implementar los conocimientos adquiridos en el transcurso del año, como manejo de archivos, registros, manejo de memoria, instrucciones, sentencias (de bucle, de asignación), entre otras.
Durante el proceso de desarrollo nos topamos con ciertos desafíos o errores que fuimos detectando y mejorando para obtener un mejor resultado, como por ejemplo el exceso de memoria (desbordamiento), funciones repetidas, bucles infinitos, etc. Pero también investigamos, aprendimos nuevos conceptos, conocimientos, técnicas para desarrollar correctamente el trabajo práctico, utilizamos distintas herramientas y fundamentalmente compartirmos ideas, estrategias, para hacer la mejor versión posible.

- ☐ En conclusión, este proyecto nos permitió aprender, trabajar en equipo (como la división de tareas, manejo de tiempos, debatir sobre datos no explícitos en las consignas), investigar nuevas herramientas como por ejemplo HTML o como reforzar conceptos ya adquiridos y tener una experiencia de cómo podría ser un proyecto en un trabajo.